GPU K-Means Clustering

Our approach, dimension-sweep results and profiling findings

GPU Programming Project | 20 October 2026

Team members: Yekkirala Venkata Radhe Shyam (G25AIT2134)
Momin Mohd Asif Mohd Naeem (G25AIT2063)
Anil Kumar Das (G25AIT2009)

Code: github.com/aniliter-cloud/kmeans-gpu-grp-project
Evidence: Shared benchmark results and profiling files on Google Drive

We compare a CPU baseline with CUDA K-Means running on an NVIDIA Tesla T4. We parallelize assignment and centroid recomputation and evaluate fused dot-score assignment.

1. Our implementation approach

Method	What we implement
CPU baseline	We use Numba Lloyd iterations with a separate fixed-iteration timing baseline.
GPU Atomic	We assign one thread per point, cache centroids in shared memory, and use global atomic sums/counts before a GPU mean update.
GPU Reduction	We use shared-memory scan reductions for centroid sums and cluster counts.
Fused assignment	We fuse dot scores with argmin on transposed data and use atomic accumulation. We update centroid means on the CPU.

We write our CUDA kernels in Python using **Numba CUDA** rather than directly in C++. Numba provides the kernel, shared-memory, synchronization and atomic functionality our implementation needs, and our team is already familiar with Python syntax.

We use $\mathbf{||x-c||^2 = ||x||^2 - 2(x \cdot c) + ||c||^2}$ in the fused path. We omit the point norm when selecting the nearest centroid and avoid an N×K score matrix. We distinguish this custom CUDA kernel from our separate CuPy/cuBLAS assignment experiment.

2. Our experiment and comparison contract

We generate clustered data with **faker_data_synthesizer.py**, prepare shared initial centroids, and upload both to Drive for Colab. We evaluate N = 10k, 50k, 100k, 1M, 5M and 10M, with three K values at each size (3 through 17). For each source file we retain every row and its K, select the first D = 2, 4 or 9 named features, and exclude ID and true-label metadata. We cover **18 source datasets and 54 cases per method**.

We use matching columns from the same full-9D canonical centroid rows, float64 data/sums, and at most 400 iterations. We stop below a centroid-movement tolerance of 10^{-4}, or at a changed-assignment fraction of at most 10^{-4} for three consecutive iterations. We retain previous centroids for empty clusters and exclude data loading and JIT warm-up from convergence timing.

3. What we validated

We find matching initialization hashes, iteration counts and stopping reasons in **54/54 cases across all four methods**. We observe a maximum final-centroid coordinate difference below $\mathbf{7.7 \times 10^{-11}}$. We find identical GPU cluster-size vectors and changed-count histories in every case; the largest relative GPU WCSS difference is below $\mathbf{2.8 \times 10^{-15}}$. We also reproduce the earlier fixed-9D numerical outputs wherever available.

We identify an output-stage difference: CPU reassigns labels against the final centroids after stopping, whereas GPU exports retain the last iteration's labels. We therefore see different CPU/GPU cluster sizes in 35/54 cases, with a maximum relative WCSS difference of about 5.6×10^{-9}. We confirm the explanation by streaming two complete cases totaling 10.1M rows; the final reassignment changes 909 labels in the 10M/K=17/D=2 case.

Results and profiling

We separate convergence runtime, GPU kernel cost and export conventions.

4. Our convergence runtime results

We show the recorded convergence seconds for **N=10,000,000 and K=17**. We use CPU convergence time here, not its repeated fixed-iteration median.

D	Iterations	CPU (s)	Atomic (s)	Reduction (s)	Fused (s)
2	100	80.032	2.995	4.674	2.951
4	104	107.218	3.972	5.957	3.640
9	183	369.043	14.281	17.944	11.243

We measure 11.243 s for Fused versus 14.281 s for Atomic in the D=9 example, a 1.27x recorded time ratio. Across the complete sweep, however, we find Fused faster than Atomic in **15/54 cases** and faster than Reduction in **22/54**. We do not claim a universal winner. We also find that lower D can require more iterations: 10M/K=13 takes 231, 42 and 185 iterations at D=2, 4 and 9.

We preserve implementation-specific timing scopes: Fused includes host mean updates, small transfers and its final label return; Atomic/Reduction exclude the final label download. We report single recorded GPU convergence runs, so these ratios do not quantify run-to-run variability. We compare WCSS only within the same feature space.

5. What our Nsight Systems captures show

We examine six SQLite exports with real CUDA activity on Tesla T4. We isolate warm-up and full-size phases for **500k/K=11 and 1M/K=17 at D=9**. We treat these as separate profiling workloads, not measurements of every dimension-sweep case. We select 60 and 129 full-size iterations, respectively.

Clustering phase	500k / K=11	1M / K=17
Atomic	5.368 ms	7.932 ms
Reduction	4.564 ms	7.451 ms
Fused + global atomic sums	3.694 ms	5.554 ms
Fused + shared atomic sums	2.337 ms	5.366 ms

We report **mean summed GPU kernel time per selected iteration** above, excluding host work and gaps. We find Atomic accumulation uses approximately **54-62%** of selected kernel time. We observe the clearest benefit from shared atomic accumulation at 500k/K=11. We also see repeated allocation and small host/device operations in the fused path, which motivate persistent buffers and GPU-resident mean updates.

We identify the cuBLAS calls in the mixed GEMM captures as standalone assignment tests, not GEMM calls on every clustering iteration. We do not add synchronization API durations to kernel execution time or infer measured occupancy/bandwidth from these traces. We infer phase boundaries from launch geometry and ordering because application NVTX ranges and original standalone capture drivers are absent.

6. Our contribution and AI assistance

We built the initial bare-bones version ourselves. We then used AI tools, including **GPT and Claude**, to clean up code, reduce boilerplate and explore efficiency improvements. We also used AI to articulate and advocate our ideas, compare alternatives and support our discussions towards team consensus. We estimate that AI-assisted work accounted for **approximately 30% of our total effort**.

We retain an export caveat: our local download lacks one Reduction summary row/assignment file (10M/K=17/D=9), 13 CPU histories and CPU assignment exports. We have all 216 final-result records; we distinguish missing exports from failed runs. We currently save no Fused per-point CSVs. We document export and reproducibility details in the design notes.